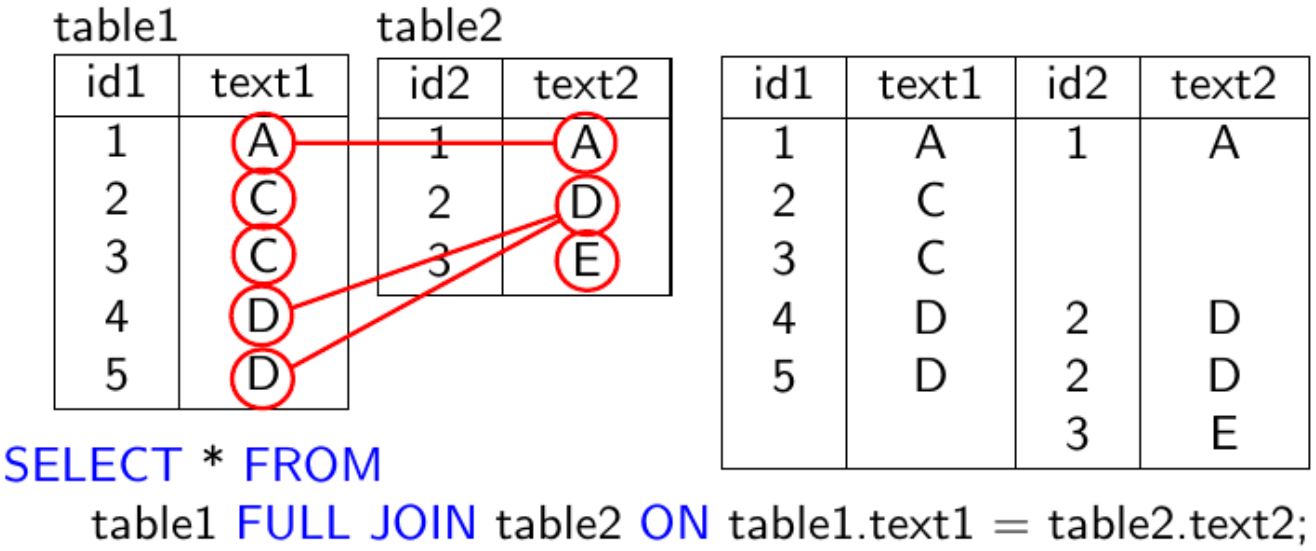
Lecture 3: Joins and Spatial Queries

Joining Multiple Tables

- Combine multiple tables into one
  - Upon ID
- 


```
SELECT * FROM
    table1 FULL JOIN table2 ON table1.text1 = table2.text2;
```

  - JOIN/INNER JOIN - For each row of table 1 a value exists in table 2
    - (3 elements)
  - LEFT JOIN - +Also rows of table 1 without correspondance in table 2
    - (5 elements)
  - RIGHT JOIN - +Also rows of table 2 without correspondance in table 1
    - (4 elements)
  - FULL JOIN - +Rows of both table - also without correspondance (table 3)
    - (6 elements)

- Which orders went to the street named "Parkstrasse"?

  ```
  SELECT orders.id FROM
  (orders JOIN customers ON orders.customerid = customers.id)
  WHERE customers.address = 'Parkstrasse';
  ```

  alternative form:

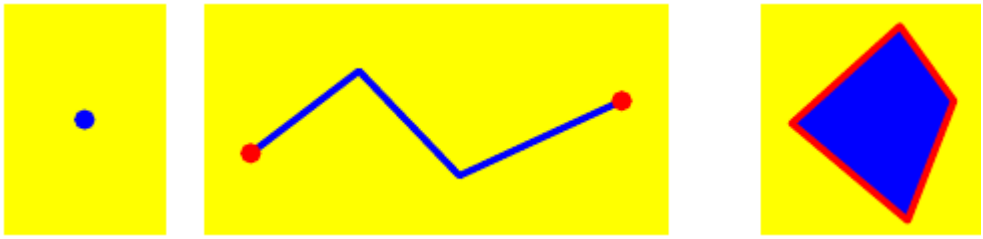
  ```
  SELECT orders.id FROM orders, customers
  WHERE orders.customerid = customers.id AND
  customers.address = 'Parkstrasse';
  ```

  - Alternative is more readable - Results are the same, execution time similar

Spatial Relationships

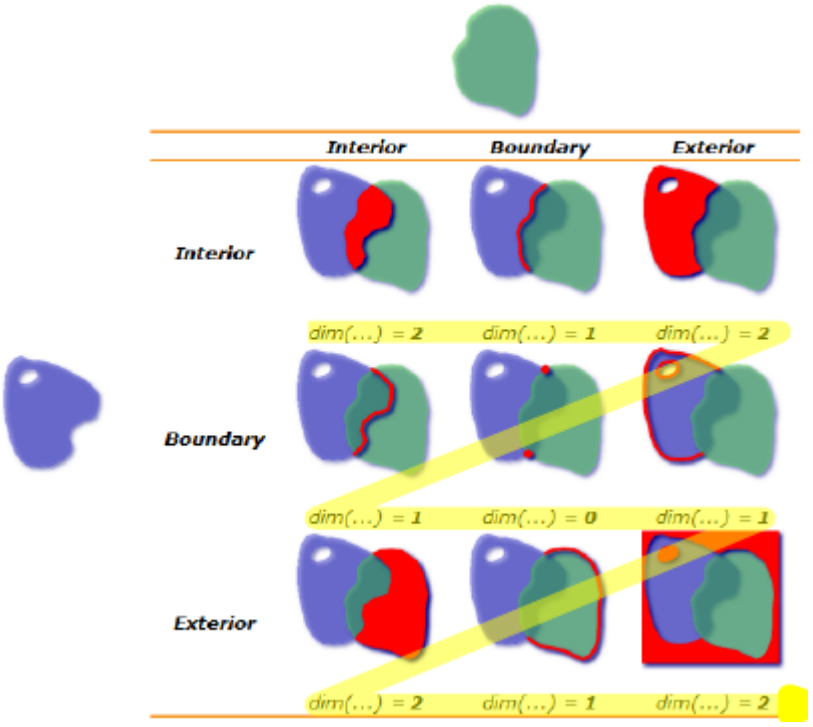
- Spatial queries are of fundamental importance for geographic applications
- Formal model of spatial relationships allow such spatial queries

Mathematical Model

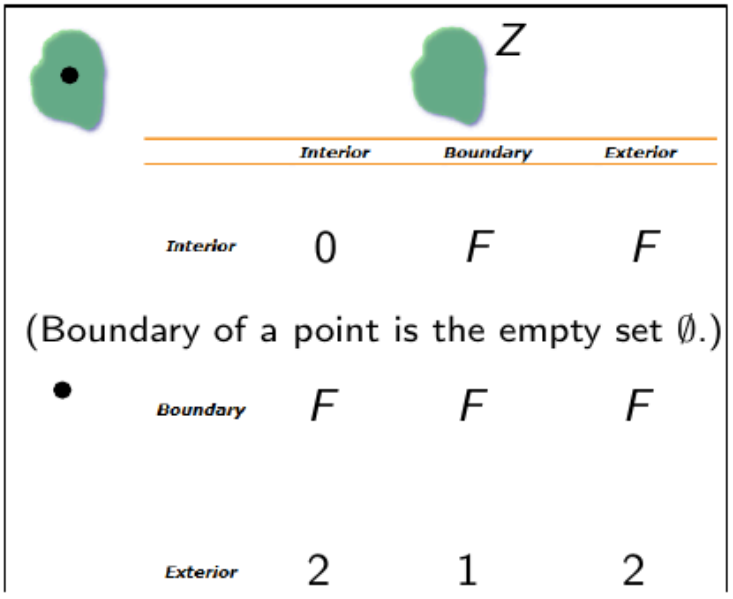
- Geometry is a set $S \subseteq \mathbb{R}^k$
  - Union: $S_1 \cup S_2$
  - Intersection $S_1 \cap S_2$
  - Difference $S_1 \setminus S_2$
- Points, Lines, Areas are **closed sets**
  - Boundary is part of the geometry S
- Interior of a geometry S is $Interior(S) = S \setminus Boundary(S)$
- Exterior of a geometry S in $\mathbb{R}^k$ is $Exterior(S) = \mathbb{R}^k \setminus S$
- 

Dimensionally Extended Nine-Intersection Model (DE-9IM)

- Standard according to OpenGIS Simple Features Implementation Specification for SQL
- Characterisation of spatial relationships based on **dimensions** of intersections of interiors, boundaries, exteriors

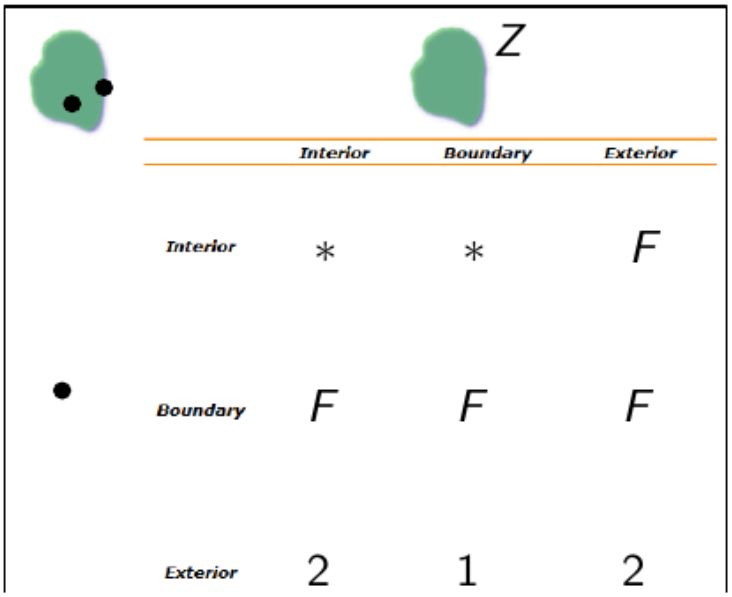
- 

- (As a single string: '212101212')
  - 0 $\longrightarrow$ point
  - 1 $\longrightarrow$ line
  - 2 $\longrightarrow$ area
  - T $\longrightarrow$ point, line, or area
  - F $\longrightarrow$ empty set
  - $*$ $\longrightarrow$ don't care
    - Which customers (points) live in zone Z (polygon)?



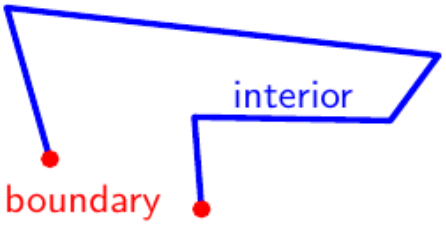
- SELECT customers.id FROM customers, zones WHERE zones.id = 1 AND ST_Relate(customers.geom, zones.geom, '0FFFFF212');

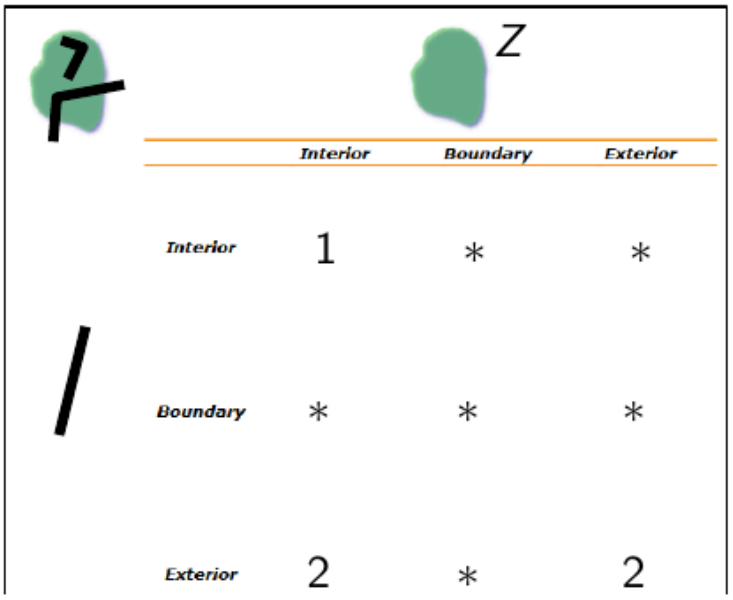
- Which customers (points) live on boundary of or in zone Z (polygon)?




```
SELECT customers.id FROM customers, zones WHERE zones.id = 1 AND
ST_Relate(customers.geom, zones.geom, '**FFFF212');
```

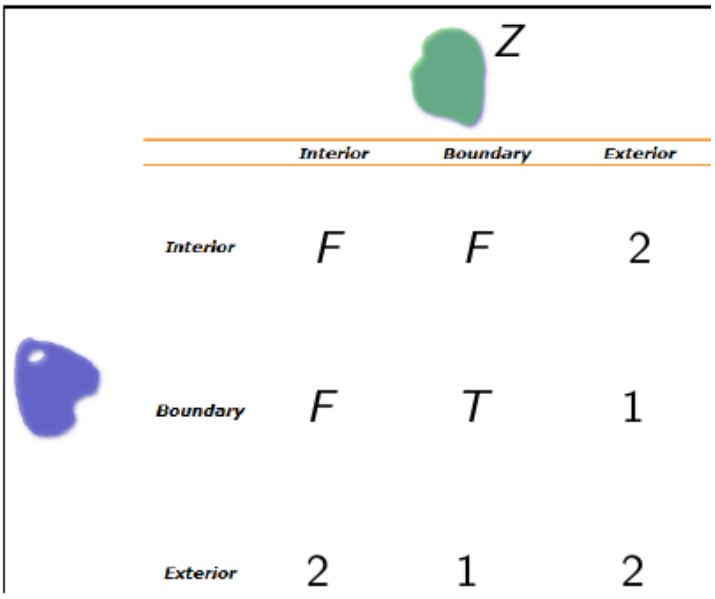
- Which roads (lines) are at least partly in Z?





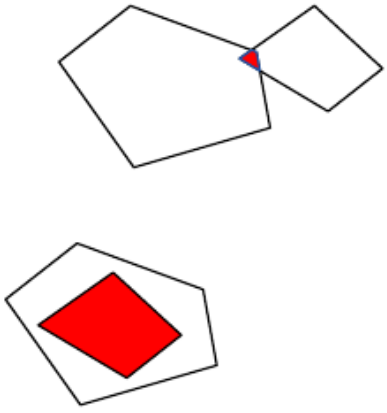

```
SELECT roads.id FROM roads, zones WHERE zones.id = 1 AND
ST_Relate(roads.geom, zones.geom, '1*****2*2');
```

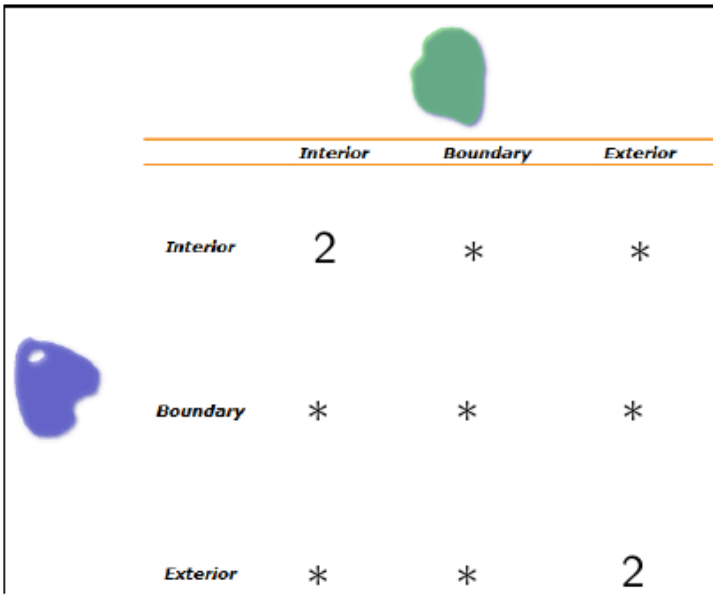
- Which zones (polygons) are adjacent to Z?



- SELECT zones1.id FROM zones AS zones1, zones AS zones2 WHERE zones2.id = 1 AND ST_Relate(zones1.geom, zones2.geom, 'FF2FT1212');

- Find all pairs of zones whose interiors intersect.



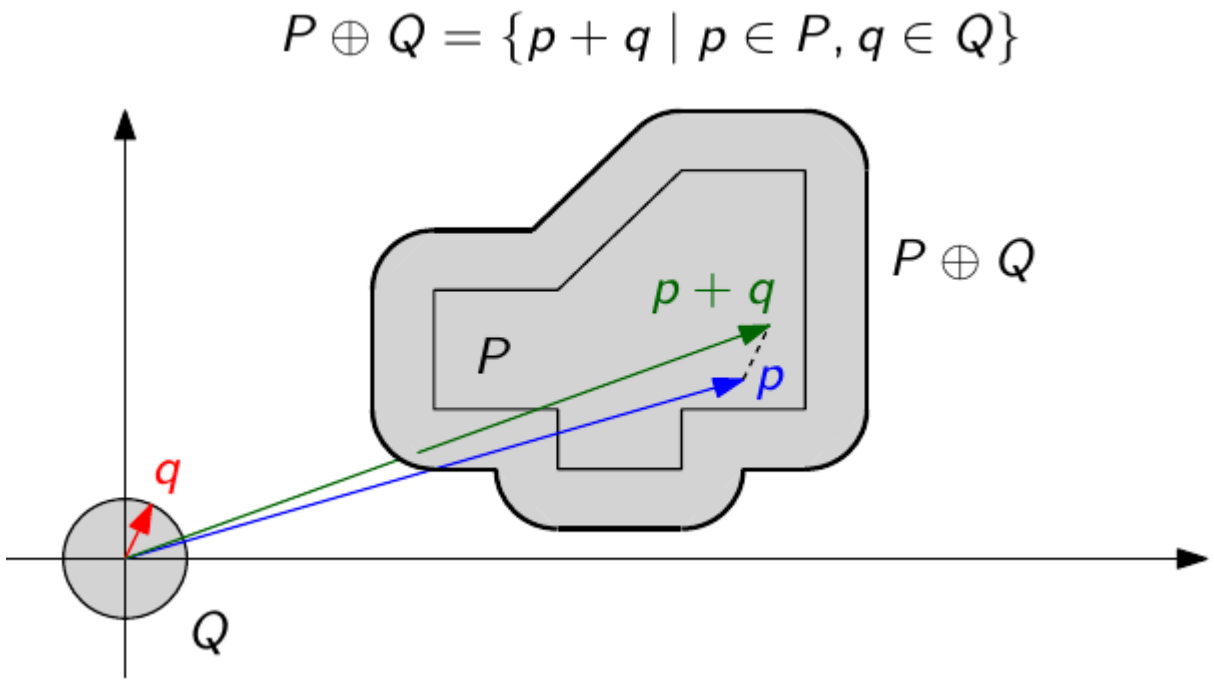


- SELECT z1.id, z2.id FROM zones AS z1, zones AS z2 WHERE ST_Relate(z1.geom, z2.geom, '2*******2');
- Exteriors of two polygons always intersect
- Boundary of a point is the empty set $\emptyset$
- Some relations entail other:
  - Two polygons p_1, p_2 it holds: $Boundary(p_1) \cap Interior(p_2) \neq \emptyset \rightarrow Interior(p_1) \cap Interior(p_2) \neq \emptyset$
- Some spatial relationships have names:
  - Equals
  - Disjoint

    - Touches
    - Crosses
    - Within
    - Overlaps
    - Contains
- Special SQL-Methods for these (usually faster)

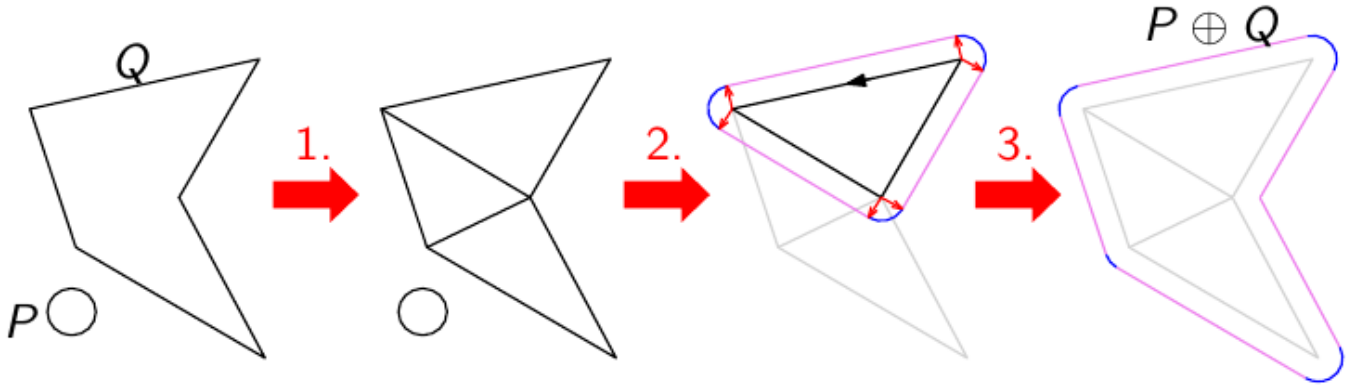
Buffers

Minkowski Sum

- Generally: A sum for two sets whose elements can be summed
    - $P \oplus Q = \{p + q | p \in P, q \in Q\}$
        - Example $\{1, 7, 9\} \oplus \{3, 4\} = \{1 + 3, 7 + 3, 9 + 3, 1 + 4, 7 + 4, 9 + 4\} = \{4, 10, 12, 5, 11, 13\}$
- Given: 2 sets $P, Q \subseteq \mathbb{R}^2$
- **Minkowsky Sum** $P \oplus Q$ contains all position vectors that result from a sum of two vectors
    - 

Minkowski Sum - Algorithm

1. Partition both P, Q into *convex* pieces (e.g., triangles, disks)
2. For each piece p of P and q of Q, compute $p \oplus q$
3. Compute the union of these sums



SQL Queries

`geometry ST_Buffer(geometry g1, float r);`

- Return g_1 buffered by distance r.

`geometry ST_MinkowskiSum(geometry g1, geometry g2);`

- Returns $g_1 \oplus g_2$.

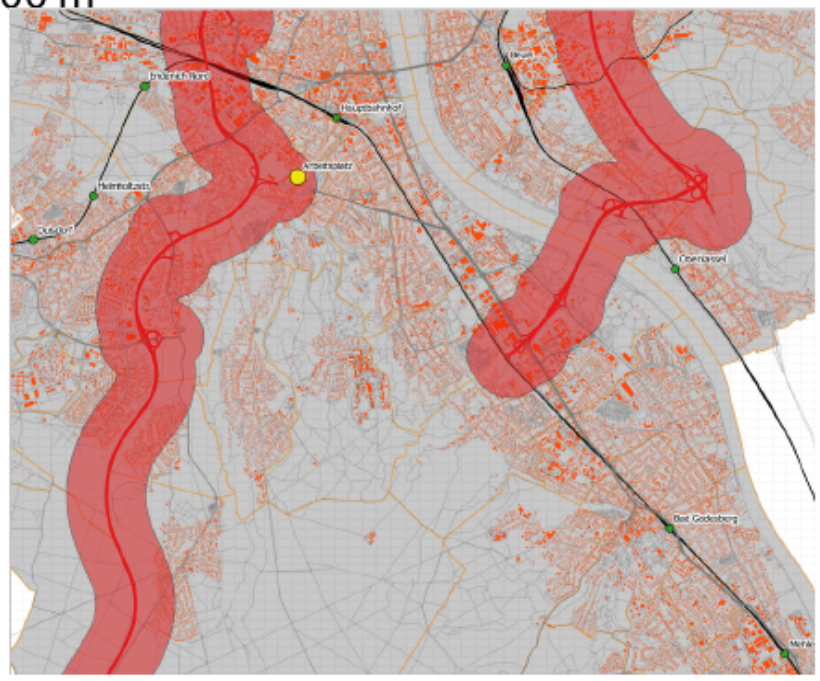
`geometry ST_DWithin(geometry g1, geometry g2, float r);`

- Returns true, if distance between geometries is at most r
  - More efficient than computiong buffers and testing for intersection

Use Case: FInding an Apartment

No Noise

- The appartment has to be at least 500 m away from the next Autobahn.
- Compute Buffer of 500 m for Autobahn features
- Minkowski Sum: $L' = L \oplus K(500)$



set of points with noise

set of points of Autobahn

disk: radius x meter, origin (0,0)